EXAM-ORIENTED STUDY GUIDE

Built-in Operators & Functions in C

Console-Based Input / Output

`printf()` • `scanf()` • `getch()` • `getchar()`
`putchar()` • `gets()` • `puts()`

A Complete Revision Resource for University Exams, Competitive Exams & Teacher Recruitment Exams

Built-in Operators and Functions — Foundational Idea

Built-in operators are foundational symbols or keywords in programming languages (like Python, Java, and C++) that instruct the compiler or interpreter to perform specific mathematical, logical, or data-handling tasks. They process variables and values to yield a final result. In C, built-in **I/O functions** work alongside these operators as ready-made tools provided by the language's standard library to handle the movement of data between the user and the program — covered in detail in this guide.

1 Introduction to Console-Based Input/Output

1.1 What is Console I/O?

Console-based Input/Output refers to the exchange of data between the user and a C program through the **computer screen (monitor)** for output and the **keyboard** for input. Since C runs in a text-based console/terminal environment, all communication happens character-by-character or string-by-string through standard streams.

1.2 Standard Input, Output and Error Streams

C automatically opens three standard streams when a program starts. They are declared in `stdio.h`:

Table 1.1 — Standard Streams in C

Stream	Stands For	Default Device	Used By
`stdin`	Standard Input	Keyboard	`scanf, gets, getchar, getch`
`stdout`	Standard Output	Monitor	`printf, puts, putchar`
`stderr`	Standard Error	Monitor	Error/diagnostic messages

1.3 Importance of I/O Functions in C

- They allow a program to **interact dynamically** with the user instead of using only hard-coded values.
- They enable **data validation**, menu-driven programs, and real-time processing.
- They form the foundation for understanding more advanced I/O (file handling, buffers, streams).

1.4 Header Files Required

Table 1.2 — Header Files for Console I/O

Header File	Purpose	Functions Declared
`<stdio.h>`	Standard Input/Output	`printf, scanf, gets, puts, getchar, putchar`
`<conio.h>`	Console Input/Output (Turbo C / Windows specific, non-ANSI)	`getch, getche, clrscr`

Exam Tip: `conio.h` is **not part of standard ANSI C**; it is supported mainly by Turbo C / Borland / MinGW-based compilers on Windows, not by GCC on Linux without modification.

1.5 Flow of Data Between User and Program

Input flows from the **keyboard → input buffer → variable in memory**, while output flows from a **variable/value → output buffer → monitor screen**.

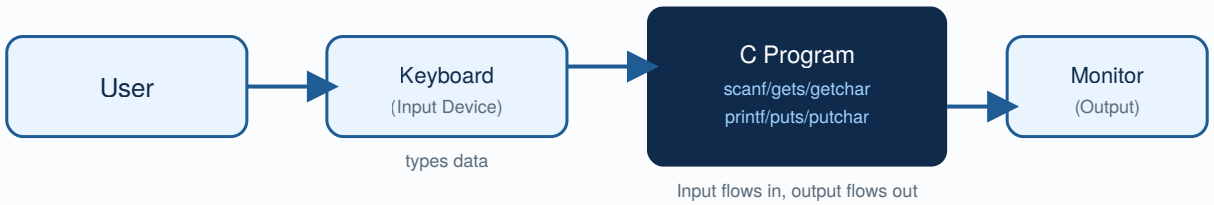


Fig. 1.1 — Console Input/Output Data Flow: User → Keyboard → Program → Monitor

Chapter Summary

- Console I/O = data exchange via keyboard (input) and monitor (output).
- Three standard streams: `stdin`, `stdout`, `stderr`.
- `stdio.h` is mandatory for all standard I/O functions; `conio.h` is compiler-specific (non-ANSI), needed only for `getch()`.

2 The printf() Function

2.1 Definition and Purpose

`printf()` is a **standard library output function** defined in `stdio.h` that displays formatted data — text, numbers, characters — on the console (standard output).

2.2 Syntax

```
SYNTAX
int printf(const char *format, ...);
```

Component	Description
`format`	A string containing plain text, format specifiers, and/or escape sequences
`...`	Variable number of arguments matching the format specifiers
Return value	Number of characters printed (negative value on failure)

2.3 Format Specifiers

Format specifiers are special codes starting with `%` that tell `printf()` (or `scanf()`) the **data type** of the value being printed or read, so it can interpret and display the bits correctly.

Table 2.1 — Common Format Specifiers

Specifier	Data Type	Example
`%d` / `%i`	int	`printf("%d", 25);`
`%f`	float	`printf("%f", 3.14);`
`%lf`	double	`printf("%lf", 3.14159);`
`%c`	char	`printf("%c", 'A');`
`%s`	string	`printf("%s", "Hello");`
`%u`	unsigned int	`printf("%u", 10);`
`%ld`	long int	`printf("%ld", 100000L);`
`%x` / `%o`	hexadecimal / octal	`printf("%x", 255);`
`%p`	pointer address	`printf("%p", &x);`
`%%`	literal percent sign	`printf("%%");`

2.4 Escape Sequences

Escape sequences are special character combinations starting with a backslash `\` that represent non-printable or special actions (like a new line or tab) inside a string.

Table 2.2 — Common Escape Sequences

Sequence	Meaning
`\n`	New line
`\t`	Horizontal tab
`\\`	Backslash
`\"`	Double quote
`\'`	Single quote
`\b`	Backspace
`\r`	Carriage return
`\0`	Null character

2.5 Examples with Output

Example 2.1

```
#include <stdio.h>
int main() {
    int age = 21;
    float marks = 89.5;
    printf("Age: %d\nMarks: %.1f\n", age, marks);
    return 0;
}
```

Output

```
Age: 21
Marks: 89.5
```

2.6 Common Errors and Best Practices

Common Errors

- Mismatched format specifier and variable type (e.g., `%d` for a float) → undefined/ garbage output
- Missing arguments for given specifiers
- Forgetting `\n`, causing output to run together

Best Practices

- Always match specifier with variable's data type
- Use field width/precision (`%5d`, `%.2f`) for aligned output
- Use `%%` to print a literal % sign

Chapter Summary

- `printf()` → formatted output function; returns number of characters printed.
- Format specifiers must match argument data types exactly.
- Escape sequences control formatting like new lines and tabs.

3 The scanf() Function

3.1 Definition and Purpose

`scanf()` is a standard library **input function** defined in `stdio.h` that reads formatted data from the keyboard (standard input) and stores it into program variables.

3.2 Syntax

```
SYNTAX
int scanf(const char *format, ...);
```

Component	Description
`format`	String containing format specifiers describing the type of input expected
`...`	**Addresses** of variables where input is stored
Return value	Number of items successfully read (EOF on input failure)

3.3 Reading Different Data Types

Example 3.1 — Reading int, float, char

```
int a; float b; char c;
scanf("%d %f %c", &a, &b, &c);
```

3.4 Use of Address Operator (&)

`scanf()` needs the **memory address** of a variable (not its value) so it can directly write the input into that memory location. This is why the **address operator &** is used before each variable name (except for strings/arrays, which already decay to an address).

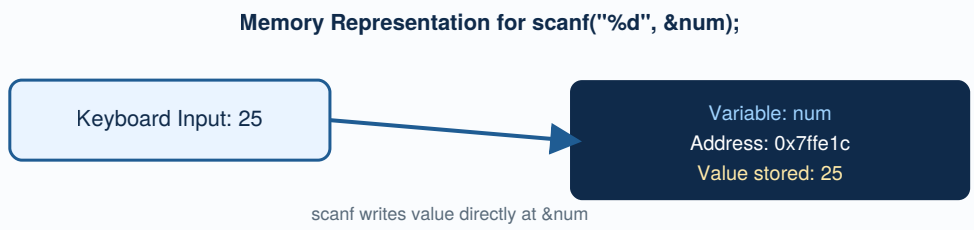


Fig. 3.1 — scanf() stores input directly into the variable's memory address using &

3.5 Examples with Output

Example 3.2

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("You entered: %d\n", num);
    return 0;
}
```

Output (user types 25)

```
Enter a number: 25
You entered: 25
```

3.6 Common Mistakes and Best Practices

Common Mistakes

- Forgetting & before a variable name
- Using `%d` with `scanf("%s",...)`-style logic for strings (no & needed for arrays)
- Leaving leftover `\n` in the input buffer before `getchar()/gets()` calls

Best Practices

- Always check `scanf()`'s return value for input validation
- Add a leading space before `%c` (`" %c"`) to skip stray whitespace/newline
- Avoid `scanf("%s",...)` for strings with spaces — it stops at the first space

Exam Tip: `scanf()` returns the number of fields successfully scanned, e.g., `scanf("%d%d", &a, &b)` returns 2 if both integers were read correctly.

Chapter Summary

- `scanf()` reads formatted input and needs variable **addresses** (&).
- Returns number of successfully read items.
- Stops reading strings at whitespace — a key limitation.

4 The getch() Function

4.1 Definition and Purpose

`getch()` reads a **single character** from the keyboard **without** displaying it on screen and **without** requiring the Enter key to be pressed. It is declared in `conio.h`.

4.2 Syntax

```
SYNTAX
int getch(void);
```

Component	Description
Parameters	None
Return value	ASCII code of the key pressed (as `int`)
Header required	`conio.h`

4.3 Features

- **Unbuffered** input — character is read the instant the key is pressed.
- **No echo** — typed character does **not** appear on screen.
- **No Enter key needed** to confirm input.

4.4 Working Mechanism

When `getch()` executes, the program pauses and waits for a single keypress. As soon as a key is pressed, its ASCII value is captured and returned immediately — the key is consumed silently from the keyboard buffer.

4.5 Examples

Example 4.1

```
#include <stdio.h>
#include <conio.h>
int main() {
    char ch;
    printf("Press any key to continue...");
    ch = getch();
    printf("\nYou pressed (hidden): %c\n", ch);
    return 0;
}
```

Output (user presses 'A')

```
Press any key to continue...
You pressed (hidden): A
```

Note: the pressed key 'A' is **not echoed** on screen when typed — it only appears afterward because the program explicitly prints it via `printf`.

4.6 Advantages and Limitations

Advantages

- Useful for "Press any key to continue" prompts
- Ideal for password input (no echo) and menu-driven programs
- Instant single-key response, no Enter needed

Limitations

- Non-standard — not part of ANSI C; works mainly on Turbo C/Windows compilers
- Not portable to Linux/GCC without alternatives (e.g., `termios.h`)
- Cannot read more than one character at a time

Chapter Summary

- `getch()`: reads one character, no echo, no Enter required, from `conio.h`.
- Returns ASCII value of key pressed; non-portable/compiler-specific.

5 The getchar() Function

5.1 Definition and Purpose

`getchar()` is a **standard ANSI C function** (in `stdio.h`) that reads a single character from the standard input (keyboard).

5.2 Syntax

```
SYNTAX
int getchar(void);
```

Component	Description
Parameters	None
Return value	ASCII value of character read, or `EOF` on end-of-file/error
Header required	`stdio.h` (standard, portable)

5.3 Reading a Single Character

`getchar()` is **line-buffered**: it waits until the Enter key is pressed, and the typed character **is echoed** on screen as it is typed.

5.4 Examples

Example 5.1

```
#include <stdio.h>
int main() {
    char ch;
    printf("Enter a character: ");
    ch = getchar();
    printf("You entered: %c\n", ch);
    return 0;
}
```

Output (user types B and presses Enter)

```
Enter a character: B
You entered: B
```

5.5 Difference from getch()

Table 5.1 — getchar() vs getch()

Feature	getchar()	getch()
Header file	`stdio.h`	`conio.h`
Standard?	ANSI C standard, portable	Non-standard, compiler-specific
Echo on screen	Yes	No
Needs Enter key	Yes	No
Buffering	Line-buffered	Unbuffered

Chapter Summary

- `getchar()` reads one character, echoes it, and needs Enter — standard and portable.
- Main contrast with `getch()`: portability, echo behavior, and buffering.

6 The putchar() Function

6.1 Definition and Purpose

`putchar()` is a standard library function (`stdio.h`) used to display/output a **single character** to the console.

6.2 Syntax

```
SYNTAX
int putchar(int character);
```

Component	Description
`character`	The character (or its ASCII int value) to be printed
Return value	The character written (as int), or `EOF` on error

6.3 Displaying a Character

Example 6.1

```
#include <stdio.h>
int main() {
    char ch = 'Z';
    putchar(ch);
    putchar('\n');
    return 0;
}
```

Output

```
Z
```

6.4 Applications

- Printing characters one at a time inside loops (e.g., printing a string character-by-character)
- Building simple pattern-printing programs (stars, pyramids)
- Lightweight alternative to `printf("%c", ch)` for single-character output

Exam Tip: `putchar(ch)` is functionally equivalent to `printf("%c", ch)` but is slightly faster since it avoids format-string parsing.

Chapter Summary

- `putchar()` outputs exactly one character; returns the character written.
- Commonly used in loops for character-wise output.

7 The gets() Function

7.1 Definition and Purpose

`gets()` is a standard library function (`stdio.h`) that reads an entire line of text (including spaces) from standard input until the Enter key (newline) is pressed, and stores it as a string.

7.2 Syntax

```
SYNTAX
char *gets(char *str);
```

Component	Description
`str`	Pointer to a character array (buffer) where the input string is stored
Return value	`str` on success, `NULL` on failure/EOF

7.3 Reading String Input

Unlike `scanf("%s", ...)`, which stops reading at the **first whitespace**, `gets()` reads the **whole line including spaces** and replaces the trailing newline with a null terminator `\0`.

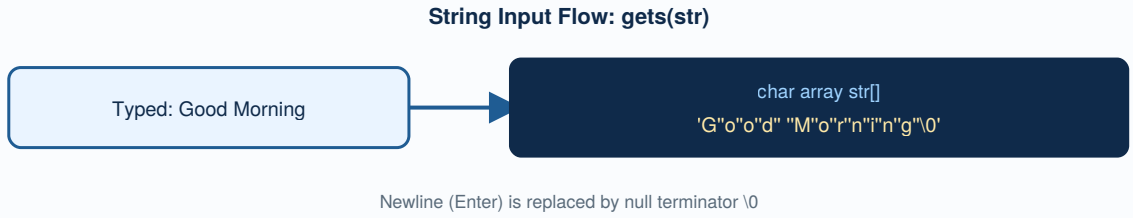


Fig. 7.1 — gets() reads spaces too, terminating the string with \0 at Enter

7.4 Examples

Example 7.1

```
#include <stdio.h>
int main() {
    char name[50];
    printf("Enter your full name: ");
    gets(name);
    printf("Hello, %s!\n", name);
    return 0;
}
```

Output (user types: Arun Kumar)

```
Enter your full name: Arun Kumar
Hello, Arun Kumar!
```

7.5 Limitations and Security Risks

Major Limitations

- **No bounds checking** — it does not check the size of the destination array, causing **buffer overflow** if input exceeds array size.
- Cannot detect or limit input length, making it a serious **security vulnerability**.
- Officially **removed from the C11 standard** due to these risks.

7.6 Safer Alternatives (Brief Note)

Modern, safer alternatives include `fgets(str, size, stdin)`, which takes a maximum buffer size and prevents overflow.

SAFER ALTERNATIVE

```
fgets(str, sizeof(str), stdin);
```

Chapter Summary

- `gets()` reads a full line with spaces but has **no bounds checking** — removed from C11.
- `fgets()` is the safer, recommended replacement.

8 The puts() Function

8.1 Definition and Purpose

`puts()` is a standard library function (`stdio.h`) that displays a string on the console followed automatically by a newline.

8.2 Syntax

```
SYNTAX
int puts(const char *str);
```

Component	Description
`str`	Pointer to the null-terminated string to be printed
Return value	Non-negative value on success, `EOF` on error

8.3 Displaying Strings

Example 8.1

```
#include <stdio.h>
int main() {
    char msg[] = "Welcome to C Programming";
    puts(msg);
    return 0;
}
```

Output

```
Welcome to C Programming
```

8.4 Difference Between printf() and puts()

Table 8.1 — printf() vs puts()

Feature	printf()	puts()
Purpose	Formatted output (any data type)	Outputs only strings
Format specifiers	Required (`%d`, `%s`, etc.)	Not used
Newline	Must be added manually (`\n`)	Added automatically
Speed	Slower (parses format string)	Faster for plain strings

Chapter Summary

- `puts()` prints a string and adds a newline automatically.

- Simpler and faster than `printf()` for plain string output, but cannot format other data types.

9 Comparison of I/O Functions

9.1 printf() vs puts()

See Table 8.1 above — `printf()` is general-purpose formatted output; `puts()` is restricted to strings with automatic newline.

9.2 scanf() vs getchar()

Feature	scanf()	getchar()
Reads	Multiple/mixed data types using format specifiers	Single character only
Address operator	Required (&) for non-array variables	Not required
Use case	General formatted input	Character-by-character input

9.3 getchar() vs getch()

See Table 5.1 above — key differences are portability, echoing, and the need for the Enter key.

9.4 puts() vs putchar()

Feature	puts()	putchar()
Outputs	Entire string	Single character
Newline	Added automatically	Not added automatically
Parameter type	`char *`	`int` (character)

9.5 Master Comparison Table

Table 9.1 — All Seven I/O Functions at a Glance

Function	Type	Header	Reads/Writes	Echo	Standard?
`printf()`	Output	stdio.h	Formatted data (any type)	—	Yes
`scanf()`	Input	stdio.h	Formatted data (any type)	Yes	Yes
`getch()`	Input	conio.h	Single character	No	No
`getchar()`	Input	stdio.h	Single character	Yes	Yes
`putchar()`	Output	stdio.h	Single character	—	Yes
`gets()`	Input	stdio.h	Whole line (with spaces)	Yes	Removed in C11
`puts()`	Output	stdio.h	String + auto newline	—	Yes

10 Programming Examples

10.1 Sample Programs Using Each Function

(Individual examples for each function are provided in Chapters 2–8. Below are programs that **combine multiple I/O functions**, as commonly asked in exams.)

10.2 Program Combining printf(), scanf(), getchar()

Example 10.1 — Student Record

```
#include <stdio.h>
int main() {
    char name[30];
    int marks;
    char grade;

    printf("Enter student name: ");
    scanf("%s", name);
    printf("Enter marks: ");
    scanf("%d", &marks);

    getchar();   /* clears leftover newline from input buffer */

    printf("Enter grade: ");
    grade = getchar();

    printf("\n--- Student Record ---\n");
    printf("Name : %s\n", name);
    printf("Marks: %d\n", marks);
    printf("Grade: %c\n", grade);
    return 0;
}
```

Output

```
Enter student name: Riya
Enter marks: 92
Enter grade: A

--- Student Record ---
Name : Riya
Marks: 92
Grade: A
```

10.3 Program Combining gets(), puts(), putchar()

Example 10.2 — Greeting Program

```c
#include <stdio.h>
int main() {
    char sentence[100];
    printf("Enter a sentence: ");
    gets(sentence);
    puts("You entered:");
    puts(sentence);
    putchar('\n');
    putchar('-');
    putchar('-');
    putchar('\n');
    return 0;
}
```

Output (user types: I love C programming)

```
Enter a sentence: I love C programming
You entered:
I love C programming

--
```

10.4 Program Using getch() for Pause

Example 10.3

```c
#include <stdio.h>
#include <conio.h>
int main() {
    printf("Calculation complete.\n");
    printf("Press any key to exit...");
    getch();
    return 0;
}
```

10.5 Common Programming Errors and Debugging Tips

Table 10.1 — Errors and Fixes

Error	Cause	Fix
Garbage value printed	Wrong format specifier used in printf/scanf	Match specifier to variable type
Program skips character input	Leftover `\n` in buffer after scanf	Insert `getchar()` or use `" %c"`
Crash / unexpected behaviour	Buffer overflow from `gets()`	Use `fgets()` with size limit
scanf("%s") stops early	Whitespace ends string read in scanf	Use `gets()`/`fgets()` for full lines
conio.h not found (GCC/Linux)	Non-standard header unsupported	Use alternatives or Turbo C/MinGW

11 Quick Revision — One-Page Notes

11.1 Important Definitions (One-Liners)

- **Console I/O:** Data exchange between user and program via keyboard/monitor.
- **printf():** Prints formatted output to the screen.
- **scanf():** Reads formatted input from the keyboard using variable addresses.
- **getch():** Reads one keypress silently, no Enter needed (conio.h).
- **getchar():** Reads one character, echoed, waits for Enter (stdio.h).
- **putchar():** Prints a single character to the screen.
- **gets():** Reads a full line including spaces; unsafe, removed in C11.
- **puts():** Prints a string with an automatic newline.

11.2 Syntax Summary Table

Function	Syntax
`printf()`	`int printf(const char *format, ...);`
`scanf()`	`int scanf(const char *format, ...);`
`getch()`	`int getch(void);`
`getchar()`	`int getchar(void);`
`putchar()`	`int putchar(int character);`
`gets()`	`char *gets(char *str);`
`puts()`	`int puts(const char *str);`

11.3 Format Specifiers Table

Specifier	Type
`%d / %i`	int
`%f`	float
`%lf`	double
`%c`	char
`%s`	string
`%u`	unsigned int
`%x / %o`	hex / octal

11.4 Common Escape Sequences

Sequence	Meaning
`\n`	New line
`\t`	Tab
`\\`	Backslash
`\"`	Double quote
`\0`	Null character

One-Page Exam Revision Checklist

- ☐ stdio.h vs conio.h — know which functions belong to which
- ☐ Format specifiers and escape sequences memorized
- ☐ Difference tables: getch/getchar, printf/puts, puts/putchar
- ☐ Why gets() is unsafe + fgets() as the safer alternative
- ☐ Role of & (address operator) in scanf()
- ☐ Practice at least one combined-function program